Lab 02 — Images, layers and registries

Theory — how images get their names, what they are made of, where they come from, and why Podman refuses to guess.

Objectives

- Break a full image name into its parts and know which parts Docker fills in silently — and which ones Podman refuses to fill in.
- Understand why a **tag** is a movable label while a **digest** is an identity.
- Explain the **layer** model and what it means for disk space and network traffic.
- Inspect an image without running it.
- Know where Docker Hub, private registries and "official" images fit in.

---

1. The full name of an image

You type `podman pull postgres:16-alpine`. The engine reads:

```
docker.io / library / postgres : 16-alpine
└registry┘  └namespace┘ └─repo──┘ └──tag──┘
```

Part	Role	Default value
registry	The server that hosts the image	`docker.io` (Docker Hub)
namespace	The organisation or user that owns it	`library` (official images)
repository	The application name	*required*
tag	The version	`latest`

Three things follow directly:

- `postgres` on its own means `docker.io/library/postgres:latest`.
- A company image carries a full name, such as `registry.mycompany.be/payments-team/billing-api:1.4.2`. A **dot** or a **port** in the first part marks it as a registry rather than a namespace.
- Images in the `library` namespace are the **official images**: maintained together with Docker, audited, and documented (`postgres`, `nginx`, `node`, `eclipse-temurin`). An image called `bobfrom59/postgres` comes with none of those guarantees.

PODMAN
Docker silently expands `postgres` to `docker.io/library/postgres`. Podman treats that as a risk: depending on which registry gets queried, a short name can resolve to a completely different image (*typosquatting*, or a namesake on an internal registry). So Podman follows `registries.conf`: it resolves known **aliases** (`alpine`, `nginx`, `postgres`…) without asking, and looks everything else up in the `unqualified-search-registries` list — and if that list holds more than one entry, it makes you choose. On Ubuntu the list contains only `docker.io`, so short names "just work"; on Fedora or a `podman machine`, they trigger a prompt. An image you build locally gets the **`localhost/`** prefix: `podman build -t api:1.0` produces `localhost/api:1.0`. `podman images` always shows the full name — no guesswork involved.

PITFALL
`latest` does not mean "the newest version". It is simply the **default** tag; the publisher may move it or leave it alone, and it can happily point at a build from two years ago. Production bans `latest`: deployments stop being reproducible, and there is nothing to roll back to.

2. Moving tag, immutable digest

A **tag** is a pointer. `postgres:16` resolves to `16.10` today and to `16.11` tomorrow; nothing changes on your disk, but the next `pull` fetches something different. A **digest** is the SHA-256 fingerprint of the manifest: `postgres@sha256:9d0d1f1e…`. Because it is **computed from the content**:

- two images with the same digest are bit-for-bit identical, wherever they live;
- an image cannot change without its digest changing too;
- you can therefore pin a deployment down exactly.

→ SECURITY
SHA-256 is a **hash function**: it maps any input to 64 hexadecimal characters, deterministically (same input, same output) and one-way (nobody can craft an input that produces a chosen output). Flip a single bit in the image and the whole fingerprint changes. Git identifies commits on the same principle: the identifier doubles as a proof of integrity.

```
podman image inspect --format '{{.Digest}}' postgres:16-alpine
podman pull docker.io/library/postgres@sha256:9d0d1f1e...   # perfectly reproducible
```

Most companies settle on the same compromise: one unique, immutable tag per build (`api:1.4.2` or `api:2026.03.17-b318`) that is never reused, plus deployment tooling that pins the digest.

3. An image is a stack of layers

Every build instruction that touches the file system produces a **layer**: the set of files it added, changed or removed relative to the previous state. The final image is those layers stacked up, plus a manifest that lists them and a configuration (default command, variables, user…).

```
┌──────────────────────────────┐  layer 4: COPY app.jar           (60 MB)
├──────────────────────────────┤  layer 3: the JRE                (180 MB)
├──────────────────────────────┤  layer 2: system packages        (30 MB)
├──────────────────────────────┤  layer 1: Debian slim base       (75 MB)
└──────────────────────────────┘

       ↑ read-only, shared between all the images that contain them
```

→ LINUX
The `overlay` storage driver is a kernel file system that **stacks** directories: read-only "lower" layers under a single writable "upper" layer. A read searches from top to bottom. A write first copies the file up into the writable layer (*copy-on-write*). A delete creates a ghost file (*whiteout*) that hides the original without removing it. Everything images do follows from those three rules.

This structure explains four behaviours you will run into constantly:

1. Disk sharing. If your twelve microservices all start from the same JRE, those 180 MB are stored **once**. Adding up the `SIZE` column of `podman images` therefore overstates disk usage by a

wide margin — `podman system df` reports the real figure.

2. Differential transfer. A `pull` or `push` only moves the layers the other side is missing: redeploying your API often transfers nothing but the JAR layer.

3. Immutability — mistakes included. If one layer adds a password and a later layer deletes it, **the file is still in the image**. The later layer merely hides it, and anyone holding the image can dig it out. A secret must never enter a build (lab 08).

4. The build cache. Layers are identified by their content, so the engine reuses any layer it already has (lab 04).

> **REMEMBER**
> Layers are shared per host or per registry, not per image: an identical layer in two images is stored once. In rootless mode that storage sits in **your** home directory (`~/.local/share/containers/storage`), so two users on the same machine share nothing.

> **PITFALL**
> A tag actually points at a **manifest list** with one entry per architecture (`linux/amd64`, `linux/arm64`…); `pull` picks the one that matches your machine. That is why an image built on an Apple Silicon MacBook refuses to start on an `amd64` server: `exec format error`. `--platform` forces the architecture.

4. Everyday commands

```
podman pull nginx:alpine                  # download without running
podman images                             # list local images
podman images --filter dangling=true      # untagged images (orphaned layers)
podman history nginx:alpine               # the layers, their size and origin
podman image tree nginx:alpine            # the layers… as a tree, with their source image
podman image inspect nginx:alpine         # full metadata as JSON
podman tag nginx:alpine my-nginx:v1       # add a name (becomes localhost/my-nginx:v1)
podman rmi my-nginx:v1                    # remove a name (and the image if it was the last)
podman system df                          # space actually used
```

Two subtleties trip people up regularly:

`podman tag` copies nothing. It sticks an extra label on the same image; both names show the same `IMAGE ID`. The reverse holds too: `rmi` on an image with two tags only removes the tag. The data disappears only when the last name does.

A "dangling" image (`<none>:<none>`) is not mysterious junk. Its tag simply moved on to a newer build: the image lost its name but still takes up disk space. That is the normal leftover of repeated rebuilds.

Getting an image out of the engine

```
podman save -o api.tar my-api:1.0                       # archive in docker-archive format
podman save --format oci-archive -o api.tar my-api:1.0  # same thing, in the standard OCI format
podman load -i api.tar                                  # re-imports the image, tags included
```

This matters when the target machine cannot reach any registry (an isolated site). Don't confuse it with `export` / `import`, which flatten the file system **of a container** and throw away both the layers and the configuration (`CMD`, `ENV`, `EXPOSE`…).

5. Registries

A registry is an HTTP service that stores layers and manifests behind a standardised API (`/v2/…`) that every tool understands.

> **→ HTTP**
> A REST API exposes *resources* at URLs and manipulates them with HTTP verbs: `GET /v2/_catalog` lists the repositories, `HEAD /v2/api/manifests/1.0` returns the digest in a header, `PUT` uploads a layer. Plain `curl` is enough to talk to a registry — you will do exactly that in the hands-on lab. A Spring Boot API runs on the same mechanics.

Type	Examples	Use
Public	Docker Hub, `ghcr.io`, `quay.io`	Base images and off-the-shelf software
Private, managed	AWS ECR, Azure ACR, Google AR	In-house images, hosted in the cloud
Private, self-hosted	Harbor, Nexus, GitLab Registry	In-house images, full control, vulnerability scanning

The typical workflow in a company:

```
podman login registry.mycompany.be
podman tag api:1.4.2 registry.mycompany.be/payments/api:1.4.2
podman push registry.mycompany.be/payments/api:1.4.2
```

Three things worth knowing:

- **`podman login` stores the token** in `${XDG_RUNTIME_DIR}/containers/auth.json`, a temporary file that vanishes when you log out. Docker writes it to `~/.docker/config.json` instead — merely base64-encoded, and it stays there for good. CI agents should use short-lived credentials.
- **Podman insists on TLS.** It refuses a plain-HTTP registry — such as the one you will run on `localhost:5000` — with `http: server gave HTTP response to HTTPS client`, until you pass `--tls-verify=false` or mark the registry `insecure = true` in `registries.conf`. Docker quietly makes an exception for `localhost`; Podman does not.
- **Docker Hub rate-limits anonymous pulls** (a per-IP quota). On CI that surfaces as `toomanyrequests`, which is why teams run a *pull-through cache* or keep internal copies of their base images.

6. In the workplace

On a Spring Boot + Angular stack:

- The **base images** (`eclipse-temurin`, `node`, `nginx`, `postgres`) get copied into the internal registry, often with `skopeo copy` — Podman's sibling tool, which copies straight from one registry to another without downloading anything. Production never pulls from the internet: quotas, availability, and control over what enters the network all argue against it.

- CI builds `registry.internal/myapp/api:<version>` and `…/web:<version>`, then pushes them; the version comes from the Git tag or the build number. A **scanner** (Trivy, Harbor, Grype) blocks any image carrying critical vulnerabilities. Deployments reference an exact version, never `latest`.

Remember

- A full name reads `registry/namespace/repository:tag`; the defaults are `docker.io`, `library` and `latest`. Podman always displays the full name and prefixes your own builds with `localhost/`.
- `latest` is not "the most recent version": it is a default tag, and production should ban it.
- A **tag** can move; a **digest** `sha256:…` identifies exact content.
- An image is a stack of **read-only layers**, shared between images and transferred differentially. A file deleted in a later layer is still in the image — never put a secret in a build.
- `tag` duplicates nothing; `rmi` removes a name first, not data. Podman insists on TLS: use `--tls-verify=false` only for a local test registry.
- `save`/`load` carry a complete image; `export`/`import` flatten a container and lose its configuration.

Vocabulary

repository: the versions of an image. — **tag**: movable label. — **digest**: immutable SHA-256 fingerprint. — **manifest**: description of the layers and configuration; a **manifest list** indexes several architectures. — **dangling image**: image that lost its tag. — **layer**: set of file changes. — **overlay**: driver that stacks the layers. — **pull-through cache**: local mirror of a public registry. — **official image**: `library` namespace on Docker Hub. — **short name**: name without a registry, resolved through `registries.conf`. — **skopeo**: copies and inspects images across registries.